AI: the Science of making machines that think like people & act like people.

Machine Learning: Function mapping b/w input A & output B.

Python: It is an interpreted General Purpose high level language with easy syntax & dynamic semantics.

Pandas: It is a data analysis library to make cleaning & analysis fast & convenient in Python.

data science: Learning about the world from data using computation. It involves exploration, inference & Prediction.

→ Loc & iloc: Calling data set: df = Pd.read_csv("link")
df

df.shape → total rows & columns

df["candidate"] → displays all candidates

df [df["Year"] == 1828] → displays all data of year 1828

df.loc[df["Party"] == "Democratic"] → displays all rows where Party democratic

df.loc[df["Result"] == "win", ["Year", "%Popular vote"]] — displays column Year & %Popular vote for which result is win

df.iloc[0:5] — display first 5 rows only

df.iloc[1:4, 0:3] — displays rows 1,2,3 & first three columns

df.iloc[0:10, [3,4]] — displays first 10 rows & columns 3 & 4

df.loc[(df["Party"] == "Democratic") & (df["Result"] == "win")]
— displays democratic parties who won elections

df.loc[[87, 25, 179], ["Year", "candidate", "Result"]]
— selects rows 87, 25, 179 & displays these columns

difference b/w loc & iloc

Loc: selects by label	iloc: selects by integers
loc: Syntax is inclusive on right hand side of slice	iloc: Syntax is exclusive on right hand side of slice

Questions

Q Retrieve all records for the state CA from babynames dataframe
— babynamesCA [babynamesCA["State"] == "CA"]

Q Retrieve all records for CA but exclude years 1990 & 2000
— babynamesCA[(babynamesCA["State"] == "CA") & (babynamesCA["Year"] != 1990) & (babynamesCA["Year"] != 2000)]

Q Find babynames in California for year 2000 & sort them in descending order
— babynamesCA[babynamesCA["Year"] == 2000].sort_values(by="count", ascending = False)

Q Select first 10 rows of babynames dataframe
babynamesCA.head(10)

Q Select first 20 rows & only include even
babynamesCA.iloc[0:22:2]

Q Retrieve rows 50 to 100 but only display rows where count is greater than 50 & only include 'State', 'Name' & 'count' columns

babynamesCA.loc[babynames["count"] > 500, ['State', 'Name', 'count']] [50:100]

Q Get row at index 100 & display it
— babynamesCA.loc[[100]]

Q Retrieve the 'Name' & 'count' columns for first 10 rows, but only include rows where name starts with 'A'

babynamesCA.loc[babynamesCA['Name'].str.startswith('A'), ['Name', 'count']].head(10)

Alternatives of Boolean Array Selection

• .isin
names = ["Bella", "Alex", "Ani", "Lisa"]
babynames[babynames["Name"].isin(names)]

• str.startswith
babynames[babynames["Name"].str.startswith("N")]

• .groupby.filter
elections.groupby("Year")
.filter(lambda sf: sf["%"].max() < 45)
.set_index("Year")
.sort_index()

• .size — babynames.size

• describe — babynames.describe — different stats will be reported

• sample — babynames.sample() — use replacement = True for replacement
for random selection of rows

• value counts — babynames["Name"].value_counts()
for each unique value in series

• unique — babynames["Name"].unique()
returns array of unique

• sort_values
babynames["Names"].sort_values()
OR
babynames.sort_values(by = "count", ascending = False)

• groupby
groupedCA = df.groupby('Name')
groupedCA

• str.len() — df['Candidate'].str.len()

Grouping

— groupedCA = df.groupby('Name')
groupedCA

— for group_name, sub_df in groupedCA:
    print(f"Group: {group_name}")
    print(sub_df)

→ Question: Find female baby name whose popularity has fallen

female_babynames = babynames[babynames["Sex"] == "F"]
female_babynames = female_babynames.sort_values(["Year", "count"])
jenn_counts_ser = female_babynames[female_babynames["Name"] == "Jennifer"]["Count"]

→ New syntax / concise summary
elections.groupby("Party").agg(max).head(10)

→ Using a lambda
df.groupby("Party").agg(lambda x: x.iloc[0])

→ using filtering — lets only keep election records where max % is less than 45%
elections.groupby("Year")
.filter(lambda sf: sf["%"].max() < 45)
.set_index("Year")
.sort_index

→ Pivot tables grouping

```
babynames_Pivot = babynames.Pivot_table(
  index="Year",  # rows (turned into index)
  columns="Sex",  # column values
  values=["Count"],  # field to process in each group
  aggfunc=np.sum,  # group operation
)
babynames_Pivot.head(6)
```

Pivot_table output

Year	Count F	Count M
1880	90994	110491
1881	91953	100737
1882	107847	113686
1883	112319	104627
1884	129020	114249
1885	133055	127101

Group by output

Year	Sex	Count
1880	F	90994
	M	110491
1881	F	91953
	M	100737
1882	F	107847
	M	113686

- Creating Table 1: babynames in 2020

```
babynames_2020 = babynames[babynames["Year"] == 2020]
babynames_2020
```

- Creating Table 2: Presidents with first names

```
elections["First Name"] = elections["Candidate"].str.split().str[0]
```

- Joining our tables

```
merged = pd.merge(left = elections, right = babynames_2020,
                  left_on = "First Name", right_on = "Name")
```

another Example of modeling

Model: $\hat{y} = \theta_0$

Loss function $= \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y})^2$

$= \frac{1}{n}\sum_{i=1}^{n}(y_i - \theta_0)^2$

Now take derivative with respect to θ_0

$= \frac{1}{n}\sum_{i=1}^{n} 2(y_i - \theta_0)(-1)$

$= -\frac{2}{n}\sum_{i=1}^{n}(y_i - \hat{\theta}_0)$

Now we want to set to 0 & find θ_0

$0 = -\frac{2}{n}\sum_{i=1}^{n}(y_i - \hat{\theta}_0)$

$= \sum_{i=1}^{n}(y_i - \hat{\theta}_0)$

$= \sum_{i=1}^{n} y_i - \sum_{i=1}^{n}\theta_0$

$n\theta_0 = \sum_{i=1}^{n} y_i$

$\theta_0 = \frac{\sum y_i}{n}$

→ Machine Learning (Simple Linear Regression)

- Model: It is an idealized representation of a system
- 3 reasons of building models: ① To explain complex phenomena ② To make accurate predictions ③ To make causal inferences about one thing causes another

The modeling process

① Choose a model

② Choose a loss function

③ Fit the model by minimizing average loss

④ Evaluate model performance

① $\hat{y} = \theta_0 + \theta_1 x$

② $L(y, \hat{y}) = (y - \hat{y})^2$ squared loss (if no outliers) ✓

$L(y, \hat{y}) = |y - \hat{y}|$ absolute loss (if outliers include)

③ $L(y, \hat{y}) = (y - (\theta_0 + \theta_1 x))^2$

$R = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$

$R(\theta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - (\theta_0 + \theta_1 x))^2$

minimize this

For θ_0 derivative $= \frac{1}{n}\sum_{i=1}^{n}(y_i - \theta_0 - \theta_1 x_i)^2$

$= \frac{1}{n}\sum_{i=1}^{n} 2(y_i - \theta_0 - \theta_1 x_i)(-1)$

$= -\frac{2}{n}\sum_{i=1}^{n}(y_i - \theta_0 - \theta_1 x_i)$

Put equal to 0

$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{\theta}_0 - \hat{\theta}_1 x_i) = 0$

$\left(\frac{1}{n}\sum_{i=1}^{n} y_i\right) - \hat{\theta}_0 - \hat{\theta}_1\left(\frac{1}{n}\sum_{i=1}^{n} x_i\right) = 0$

$\bar{y} - \hat{\theta}_0 - \hat{\theta}_1 \bar{x} = 0$

$\hat{\theta}_0 = \bar{y} - \hat{\theta}_1 \bar{x}$

For θ_1 derivative $= \frac{1}{n}\sum_{i=1}^{n}(y_i - \theta_0 - \theta_1 x_i)^2$

$= \frac{1}{n}\sum_{i=1}^{n} 2(y_i - \theta_0 - \theta_1 x_i)(-x_i)$

$= -\frac{2}{n}\sum_{i=1}^{n}(y_i - \theta_0 - \theta_1 x_i)(x_i)$

Put equal to 0 to find θ_1

$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{\theta}_0 - \hat{\theta}_1 x_i)(x_i - \bar{x}) = 0$

$\frac{1}{n}\sum_{i=1}^{n}((y_i - \bar{y}) - \hat{\theta}_1(x_i - \bar{x}))(x_i - \bar{x}) = 0$

$\frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y})(\bar{x}_i - \bar{x}) = \hat{\theta}_1 \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$

$\hat{\theta}_1 = r\frac{\sigma_y}{\sigma_x}$